

Algoritmos

Clarisse Raingeard

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy similar a Matlab y permite el trabajo con matrices :

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

```
Y=40
alpha<-30
```

Uso de datasets o base de datos internos de R

Usando el commando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos \$ despues del comando de informacion especifico y elegimos la columna de datos deseada

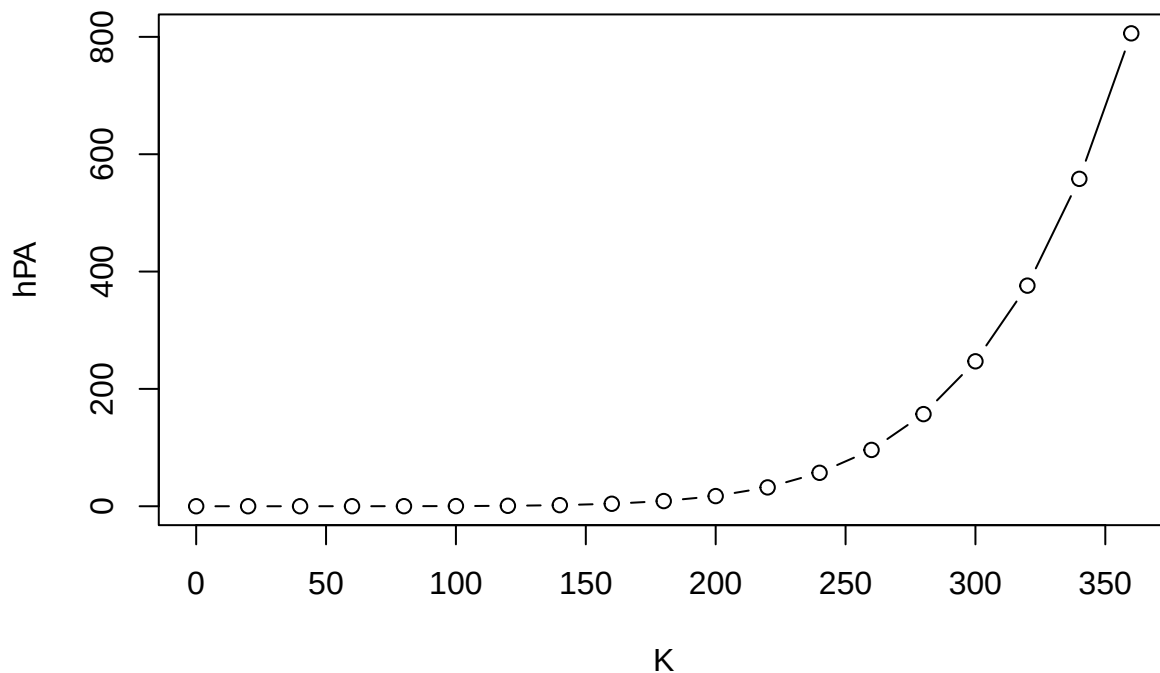
```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean    : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.    :120.00
```

Including Plots

You can also embed plots, for example:

presion del gas ideal



Commando Plot

Es un commando que permite graficar cualquier fuente de datos que le asignemos, teniendo también la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el commando podemos escribir `?plot` en la consola, lo cual abrira una ventana que explicara en detalle el comando.

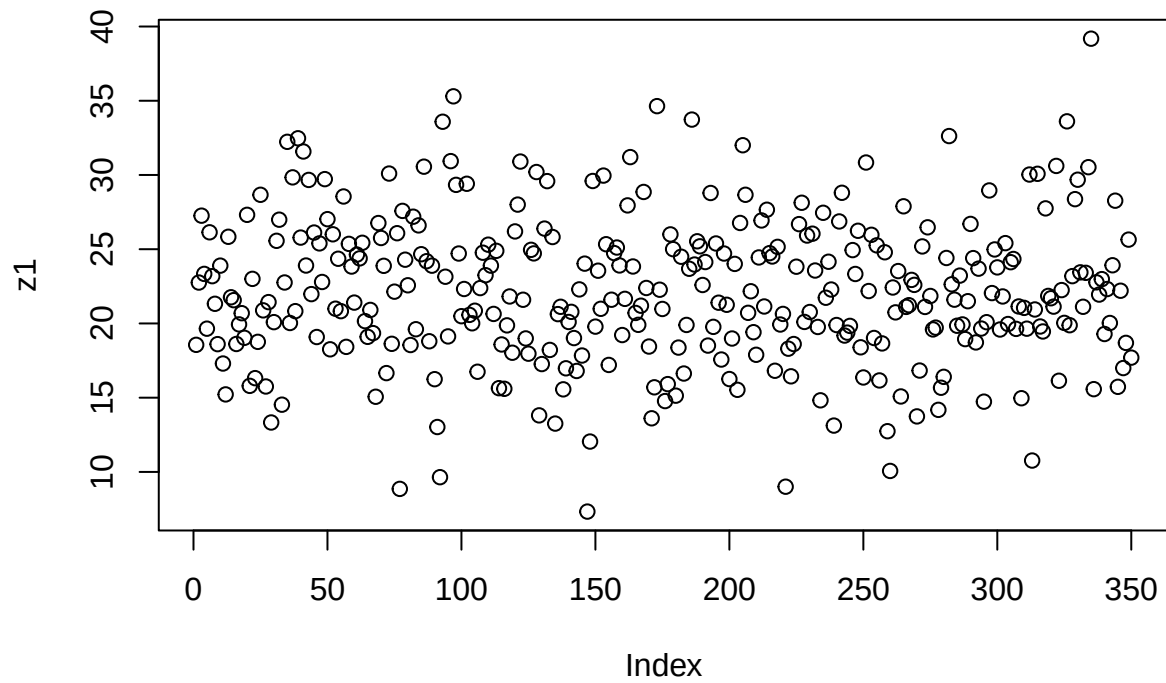
Funciones estadísticas

```
z1<-rnorm(350,22,5)
z1
```

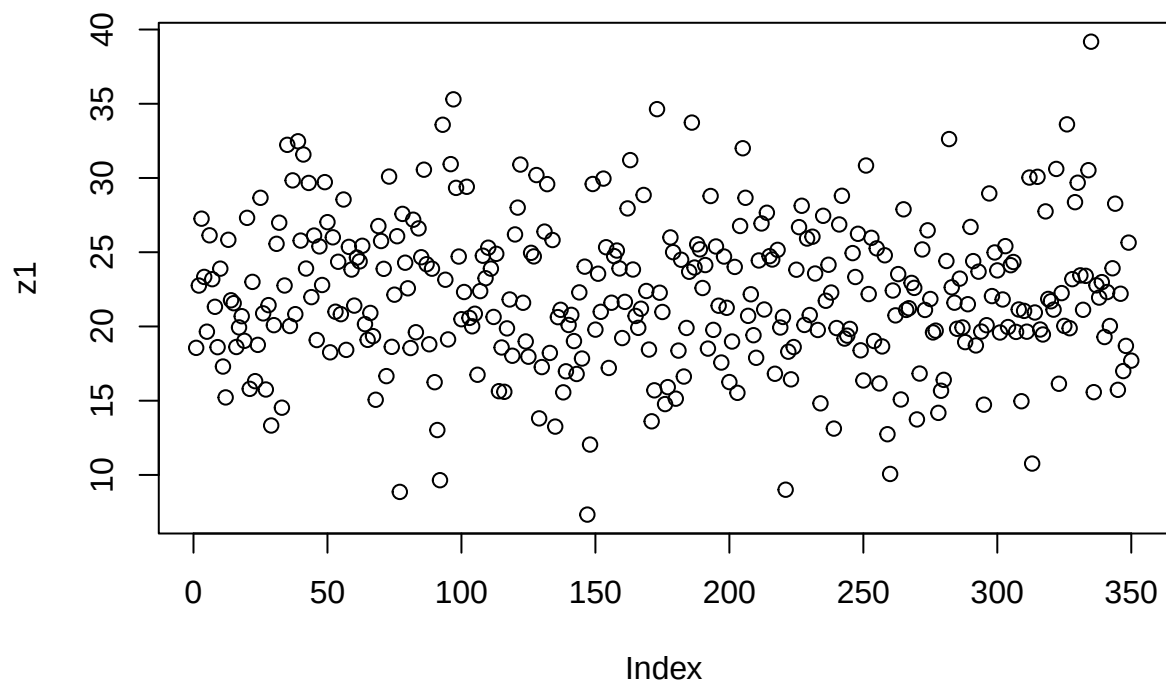
```
## [1] 18.560475 22.755169 27.260766 23.337844 19.648447 26.138705 23.184621
## [8] 21.324074 18.606994 23.893616 17.301290 15.221165 25.837339 21.763403
## [15] 21.568447 18.617366 19.936486 20.692365 19.031365 27.315791 15.796888
## [22] 23.002599 16.316473 18.761610 28.665497 20.881852 15.753450 21.436367
## [29] 13.329471 20.088475 25.566955 26.989149 14.528777 22.758807 32.233113
## [36] 20.031393 29.839626 20.828818 32.471590 25.782911 31.581643 23.901897
## [43] 29.665996 21.975859 26.122501 19.086530 25.391129 22.793369 29.719780
## [50] 27.023811 18.257422 26.005057 20.993667 24.357459 20.828128 28.551837
## [57] 18.423113 25.359288 23.833304 21.404188 24.638357 24.381694 25.433173
## [64] 20.157076 19.097762 20.910386 19.347991 15.063077 26.762160 25.754146
## [71] 23.881311 16.655650 30.096362 18.627447 22.141191 26.077220 8.857495
## [78] 27.581589 24.294787 22.570377 18.546688 27.189400 19.609784 26.603601
## [85] 24.662757 30.559781 24.192415 18.800269 23.892430 16.245832 13.023931
## [92] 9.644759 33.585857 23.147699 19.127489 30.933829 35.296803 29.333674
## [99] 24.708937 20.488016 22.320779 29.407726 20.577110 20.011659 20.857201
## [106] 16.747515 22.382245 24.764811 23.250038 25.306361 23.893208 20.641858
## [113] 24.891131 15.640482 18.584111 15.598327 19.870662 21.818136 18.028411
```

```
## [120] 26.198372 28.005698 30.904162 21.596494 18.990230 17.962866 24.959088
## [127] 24.718456 30.198329 13.808097 17.264572 26.389624 29.586207 18.217326
## [134] 25.820885 13.255333 20.628670 21.121846 15.562172 16.978912 20.098908
## [141] 20.781089 19.015137 16.799024 22.293429 17.839903 24.028258 7.329179
## [148] 12.045826 29.595449 19.785474 23.551930 20.986449 29.959461 25.335549
## [155] 17.205821 21.598413 24.740687 25.113281 23.907034 19.219224 21.656479
## [162] 27.950593 31.203503 23.837203 20.708329 19.907262 21.194585 28.856085
## [169] 22.389117 18.445115 13.611168 15.698588 34.635865 22.272431 20.975006
## [176] 14.772239 15.914027 25.998166 25.004858 15.136644 18.374803 24.494405
## [183] 16.626840 19.899735 23.684324 33.731740 23.968307 25.534782 25.194944
## [190] 22.590279 24.135926 18.510112 28.786157 19.770146 25.387505 21.397464
## [197] 17.569275 24.709945 21.257286 16.254552 18.985694 24.009906 15.530030
## [204] 26.767346 32.004355 28.666549 20.715440 22.167647 19.410427 17.889727
## [211] 24.446247 26.935776 21.142566 27.658873 24.734983 24.509260 16.813845
## [218] 25.158731 19.929097 20.642172 9.004839 18.301449 16.438278 18.613243
## [225] 23.823032 26.689903 28.129536 20.102029 25.922569 20.769875 26.046431
## [232] 23.564823 19.762501 14.822647 27.453058 21.732074 24.156785 22.284582
## [239] 13.121307 19.894529 26.869415 28.801440 19.184679 19.382925 19.835519
## [246] 24.940581 23.331252 26.240855 18.392416 16.360819 30.839187 22.183038
## [253] 25.969452 19.020397 25.261656 16.161014 18.655172 24.793546 12.742442
## [260] 10.066425 22.420741 20.745537 23.526858 15.082041 27.890142 21.115772
## [267] 21.227801 22.932868 22.597859 13.739329 16.828474 25.187849 21.112028
## [274] 26.474194 21.850415 19.598440 19.718771 14.180225 15.668819 16.403834
## [281] 24.408965 32.622020 22.636599 21.601757 19.846469 23.223071 19.933520
## [288] 18.945947 21.498801 26.703896 24.401337 18.720443 23.691896 19.657371
## [295] 14.731922 20.085019 28.959051 22.045074 24.983961 23.775065 19.593250
## [302] 21.817854 25.409489 19.960344 24.125429 24.340738 19.632971 21.144960
## [309] 14.964411 21.048466 19.648711 30.029541 10.766302 20.941185 30.080273
## [316] 19.796222 19.459665 27.750756 21.853587 21.697721 21.132904 30.609987
## [323] 16.139466 22.247696 20.037601 33.613497 19.881399 23.187060 28.372042
## [330] 29.680932 23.440683 21.122478 23.409383 30.525367 39.180092 15.574656
## [337] 22.770409 21.944665 22.985316 19.289686 22.317117 20.030802 23.918644
## [344] 28.268482 15.730759 22.204545 16.994996 18.691475 25.647234 17.697812
```

```
plot(z1)
```



```
plot(z1)
```



```
w1<-length(z1)
```

```
w1
```

```
## [1] 350
```

```
x1<-(500:849)
```

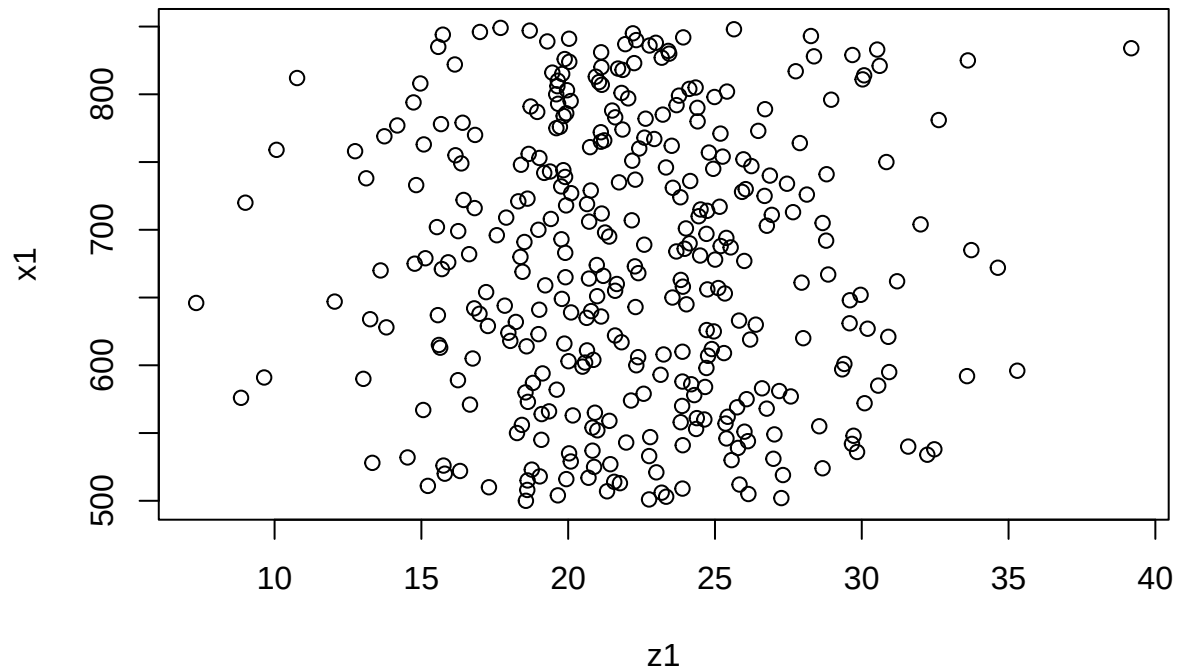
```
x1
```

```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
```

```
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
```

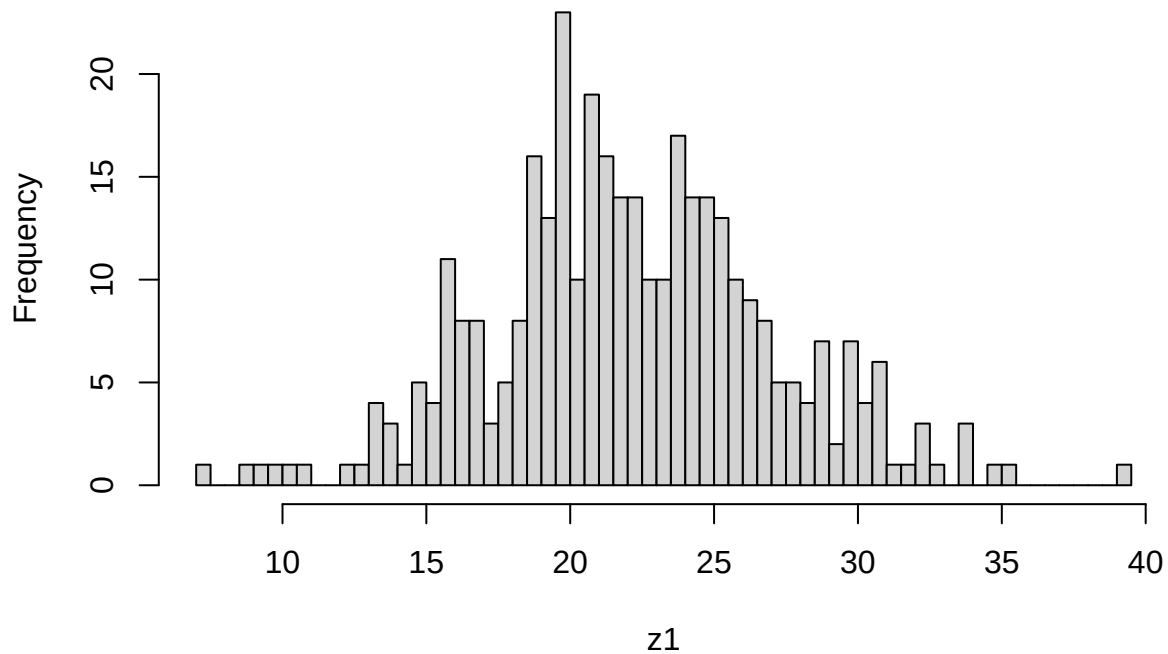
```
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(z1,x1)
```



```
hist(z1, main="histograma de edades",breaks = 50)
```

histograma de edades



```
density(z1)
```

```
##
## Call:
## density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.27
##
##      x              y
## Min.   : 3.518   Min.   :1.008e-05
## 1st Qu.:13.386   1st Qu.:1.327e-03
## Median :23.255   Median :1.043e-02
## Mean   :23.255   Mean    :2.528e-02
## 3rd Qu.:33.123   3rd Qu.:4.345e-02
## Max.   :42.991   Max.    :8.549e-02
```

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicio 1 de algoritmos

Consigna : Las ultimas 3 cifras de mi pasaporte son 219, consigna crear una variable que tenga ese numero.

```
DNI=219
```

Consigna : Crear un vector que tenga todos los numeros enteros desde el 1 al 219.

```
Secuencia_dni<-(1:219)
Secuencia_dni
```

```
##      [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
##     [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
```

```
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219
```

Ejercicio 2 :

Calcular la suma de todos los valores del vector Secuencia_dni

```
Total<-0
Valor_final<-length(Secuencia_dni)
for (i in 1:Valor_final){
Total<- Total+i
}
Total
```

```
## [1] 24090
```

Ejercicio 3

Repetir el ejercicio anterior pero en Python

```
suma=0

for i in range(1,448):
    suma+=i

print(suma)
```

```
## 100128
```

Ejercicio 4

Cuanto tarda en correr el código del ejercicio 2 ?

```
inicio<-Sys.time()
Total<-0
Valor_final<-1000000*length(Secuencia_dni)
for (i in 1:Valor_final){
Total<- Total+i
}
Total
```

```
## [1] 2.39805e+16
```

```
final<-Sys.time()
final-inicio
```

```
## Time difference of 3.500414 secs
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

Ejercicios profe:

1.4 Generar un vector secuencia

```
# Generar secuencia con bucle 'for'
# Primero inicializamos el vector
system.time({
  A <- 0
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})

##      user      system elapsed
##    0.013      0.000      0.013

# Verificamos los resultados
head(A)

## [1]  2  4  6  8 10 12

tail(A)

## [1] 99990 99992 99994 99996 99998 100000

system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})

##      user      system elapsed
##    0.001      0.000      0.000

# Verificamos los resultados
head(B)

## [1]  2  4  6  8 10 12

tail(B)

## [1] 99990 99992 99994 99996 99998 100000
```

1.5 Implementación de serie de Fibonacci

```
# Definimos la cantidad de elementos de la serie que queremos calcular
n<-30

# Método 1: Bucle 'for' con pre-asignación
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1
  for (i in 3:n) {
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```



```

    }
  })

##      user  system elapsed
##    0.003   0.000   0.003

# Ver los primeros resultados
print("Primeros números de la serie:")

## [1] "Primeros números de la serie:"
head(fib, 10)

## [1]  0  1  1  2  3  5  8 13 21 34

# Método 2: Función Recursiva

fib_recursiva <- function(k) {
  if (k == 0) return(0)
  if (k == 1) return(1)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}

print("Tiempo ejecución recursiva:")

## [1] "Tiempo ejecución recursiva:"

system.time({
  # Calculamos el elemento n de forma individual
  resultado_rec <- fib_recursiva(n)
})

##      user  system elapsed
##    0.791   0.000   0.799

```

1.6 Definición matemática recurrente

```

# --- Resolver consigna con bucle 'while' ---

# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones

system.time({
  # El bucle sigue mientras el último número generado sea <= 1.000.000
  while (fib[i] <= 1000000) {
    i <- i + 1
    # Definición recurrente:  $f(n+1) = f(n) + f(n-1)$ 
    fib[i] <- fib[i-1] + fib[i-2]
  }
})

##      user  system elapsed
##    0.004   0.000   0.004

# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")

```

```
## El primer número mayor a 1.000.000 es: 1346269
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")

## Se necesitaron 31 iteraciones (pasos) para superarlo.
# Mostramos los últimos elementos para verificar
tail(fib)

## [1] 121393 196418 317811 514229 832040 1346269
```

1.7 Ordenación de un vector del Método burbuja

```
# --- 1. Creamos un vector desordenado ---
# Usamos un vector de 1000 números aleatorios
set.seed(123) # Para que el resultado sea siempre el mismo
vector_desordenado <- sample(1:1000, 1000)

# --- 2. Implementación del Método Burbuja ---
bubble_sort <- function(x) {
  n <- length(x)
  # El bucle externo recorre todo el vector
  for (i in 1:(n - 1)) {
    # El bucle interno compara pares adyacentes
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        # Intercambio de posición (el "burbujeo")
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# --- 3. Medimos Performance del Burbuja vs R Nativo ---

cat("Tiempo con Método Burbuja:\n")

## Tiempo con Método Burbuja:
system.time({
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)
})

##      user  system elapsed
##    0.06    0.00    0.06

cat("\nTiempo con función sort() nativa de R:\n")

##
## Tiempo con función sort() nativa de R:
system.time({
  vector_ordenado_R <- sort(vector_desordenado)
})
```

```
##      user  system elapsed
##    0.000    0.000    0.001
```

1.8 La penitencia de Newton

```
# Definimos el límite de la suma (1*10^7 para que no tarde demasiado en R)
# Nota: 1*10^10 es un número extremadamente grande para un bucle for en R
n <- 1e7
```

```
# --- MÉTODO 1: La "Penitencia" (Bucle For / Fuerza Bruta) ---
# Este método suma uno por uno, como quería el profesor.
```

```
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##      user  system elapsed
##    0.164    0.000    0.163
```

```
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
# Usamos la suma de una progresión aritmética: (n * (n + 1)) / 2
```

```
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##      user  system elapsed
##         0         0         0
```

```
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

Agrupacion K-mean

Medición de Performance: Agrupamiento K-Means En el análisis de datos industriales, no basta con agrupar los componentes; es imperativo evaluar qué tan “buenas” son esas agrupaciones. A diferencia del aprendizaje supervisado, donde comparamos predicciones contra etiquetas reales, en el Clustering medimos la calidad mediante la cohesión interna y la separación externa.

1. Métricas de Evaluación Interna Para determinar si la segmentación de maquinaria tiene sentido estadístico, se utilizan principalmente dos indicadores:
 - WSS (Within-Cluster Sum of Squares): Mide la suma de los cuadrados dentro de cada grupo. Representa la variabilidad interna. Un WSS bajo indica que los componentes dentro de un cluster son muy similares entre sí (alta cohesión).
 - Coeficiente de Silueta (s_i): Es una medida que combina cohesión y separación. Su valor oscila entre -1 y 1 :
 - 1 : El componente está muy bien ubicado en su cluster.
 - 0 : El componente está en el límite entre dos clusters.
 - -1 : El componente probablemente ha sido asignado al grupo incorrecto.
2. El Gráfico de Violín como Herramienta de Validación Si bien métricas como el “Método del Codo” nos ayudan a elegir el número óptimo de clusters (k), el gráfico de violín es esencial para la interpretación física de los resultados. Este gráfico combina un diagrama de caja (boxplot) con una curva de densidad de probabilidad. En el contexto de mantenimiento industrial, nos permite visualizar:
 - Distribución y Densidad: Podemos ver dónde se concentra la mayoría de los datos de sensores (vibración, temperatura) para cada grupo.
 - Separación de Grupos: Si los “cuerpos” de los violines de diferentes clusters no se solapan significativamente, el algoritmo ha logrado

identificar estados operativos claramente distintos (ej. Estado Normal vs. Falla Inminente). Detección de Anomalías: La extensión de las “colas” del violín permite identificar máquinas que, aunque pertenecen a un grupo, presentan valores extremos que requieren atención.

```
# Librerías necesarias
library(ggplot2)
library(cluster)

# 1. Simulación de datos de sensores (Normalizados)
set.seed(123)
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
df <- data.frame(vibracion, temperatura)

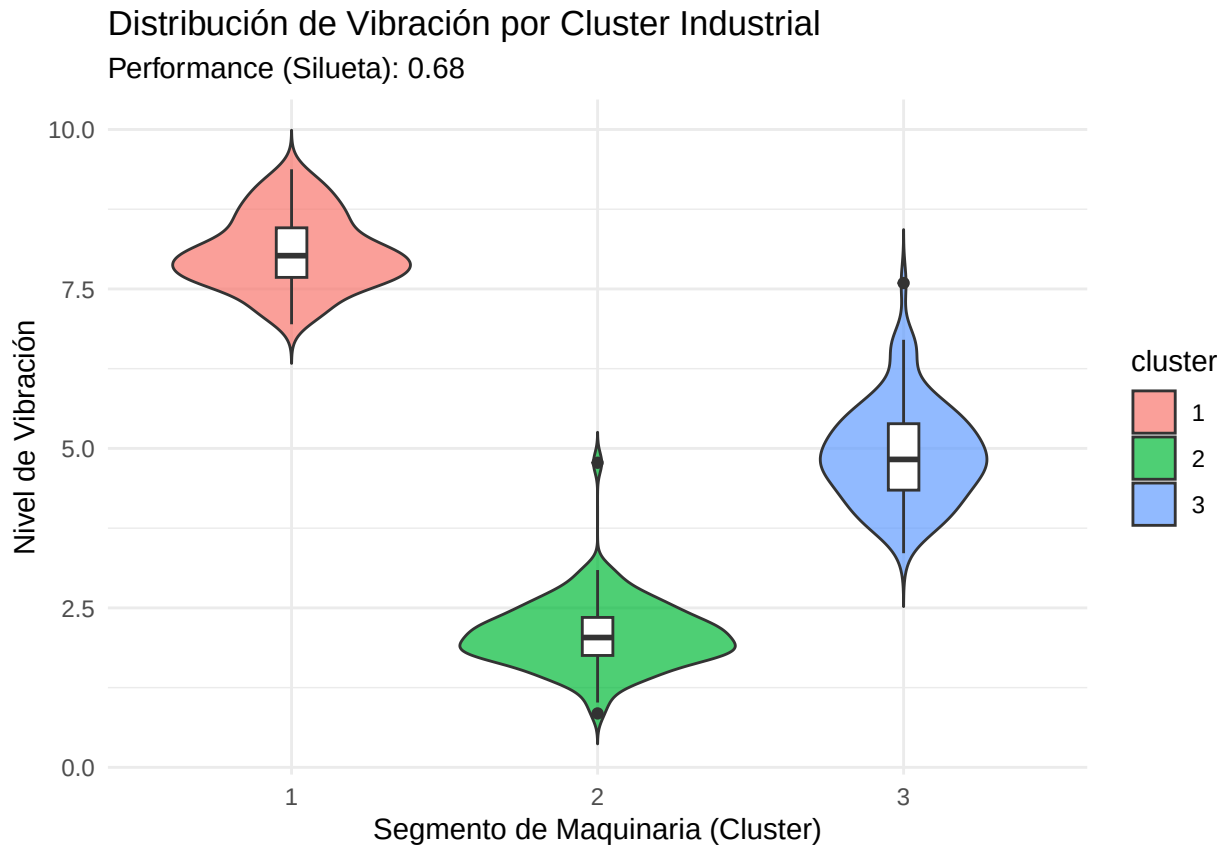
# Escalamiento (Paso crítico mencionado en tu texto)
df_scaled <- scale(df)

# 2. Ejecución de K-means (k=3)
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)

# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)

## El coeficiente de silueta promedio es: 0.6808356

# 4. Visualización con Gráfico de Violín
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
       subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
       x = "Segmento de Maquinaria (Cluster)",
       y = "Nivel de Vibración") +
  theme_minimal()
```



Matrices e Inversa

En este ejercicio, crearemos una matriz cuadrada de 3x3 utilizando el valor de tu DNI 219 y otros valores para asegurar que la matriz sea invertible (que su determinante sea distinto de cero). Luego, utilizaremos la función 'solve' que es la estándar en R para obtener la matriz inver

```
# 1. Definimos los datos de la matriz basándonos en tu DNI
# Usamos una secuencia pequeña para completar los otros campos
dni_val <- 219
```

```
datos <- c(dni_val, 10, 15,
           2, 5, 8,
           7, 1, 9)
```

```
A <- matrix(datos, nrow = 3, ncol = 3, byrow = TRUE)
```

```
cat("### Matriz Original (A):\n")
```

```
## ### Matriz Original (A):
```

```
print(A)
```

```
##      [,1] [,2] [,3]
## [1,] 219  10  15
## [2,]   2   5   8
## [3,]   7   1   9
```

```
determinante <- det(A)
```

```
cat("\nDeterminante de la matriz:", determinante, "\n")
```

```
##
## Determinante de la matriz: 7988
if (determinante != 0) {
  A_inversa <- solve(A)
  cat("\n### Matriz Inversa (A^-1):\n")
  print(A_inversa)

  identidad <- round(A %*% A_inversa, 2)
  cat("\n### Verificación (A * A^-1 = Identidad):\n")
  print(identidad)
} else {
  cat("\nLa matriz es singular y no tiene inversa.")
}
```

```
##
## ### Matriz Inversa (A^-1):
##           [,1]      [,2]      [,3]
## [1,]  0.004631948 -0.009389084  0.0006259389
## [2,]  0.004757136  0.233600401 -0.2155733600
## [3,] -0.004131197 -0.018652979  0.1345768653
##
## ### Verificación (A * A^-1 = Identidad):
##           [,1] [,2] [,3]
## [1,]      1   0   0
## [2,]      0   1   0
## [3,]      0   0   1
```